Quiz 2 - Class, object, decorator, privacy, inheritance, method, ...

Attempt 1 of 1

Written Jan 21, 2026 10:36 AM - Jan 21, 2026 10:52 AM

Attempt Score	**7 / 10 - 70 %**
Overall Grade (Highest Attempt)	**7 / 10 - 70 %**

Question 1

_____ is an instance of a _____?

- ○ Objects, Data Type
- ✓ ◉ Object, Class
- ○ Class, Object
- ○ Module, Class

Question 2

What is the purpose of the `__init__` method in a Python class?

- ○ It is a static method that is called only once when the program starts
- ○ It is automatically called every time a method of the class is accessed
- ✓ ◉ It initializes an object's attributes when a new instance of the class is created

- ○ It deletes an object and frees its memory

Question 3

What kind of a method is get_species()? (The decorator has been hidden intentionally and should not be treated as an error).

```
class Human:
    _species = "homo sapiens"

    HIDDEN DECORATOR
    def get_species(cls):
        return cls._species.title()
```

- ○ Static Method
- ✗ ◉ A regular method (your basic method)
- ○ This is not a method and will throw an error
- ➡ ○ Class Method

Question 4

Methods (excluding static and class methods), represent behaviours that belong to an object and act on the object.

- ✓ ◉ True
- ○ False

Question 5

What would be the output of the given code?

```python
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.__type = "Tabby"

    def __str__(self):
        return f"{self.name} is {self.age} years old"

my_cat = Cat("Garfield", 5)
print(my_cat._Cat__type)
```

- ✖ ◉ Attribute Error
- ➡ ○ Tabby
- ○ Garfield is 5 years old
- ○ None of the above

Question 6

class Student(Person):
  pass

In the above code, Person is a child class of Student

- ○ True
- ✔ ◉ False

Question 7

What would be the output of the given code? Note the print statement at the end.

```python
class Bird:
    def __init__(self, name, type_, color):
        self._name = name
        self._type = type_
        self._color = color


class CanadaGoose(Bird):
    def speak():
        print("I am angry and I will attack.")


my_bird = Bird("Tweety", "Canary", "Yellow")
my_canada_goose = CanadaGoose("Zorak the Destroyer", "Canada Goose", "Grey")
print(isinstance(my_canada_goose, Bird))
```

- ✔ (●) True
- ( ) False

Question 8

What would be the value of self._age if the following code was executed?

```python
class Person():
    def set_age(self, an_age):
        if an_age >= 0 and an_age <= 150:
            self._age = an_age

    def get_age(self):
        return self._age

    age = property(get_age, set_age)

    def __init__(self, name, an_age):
        self._name = name
        self.age = an_age

zorak = Person("Zorak the destroyer", 40)
zorak.age = 290
print(zorak.age)
```

- ○ 290
- ✓ ◉ 40
- ○ None
- ○ The code would throw an error.

Question 9

Which of the following is a type of formal interface in Python

- ○ Protocols
- ✓ ◉ Abstract base classes
- ○ Duck typing
- ○ Formal protocols

Question 10

What would be the output of the following code?

```python
import abc

class BankAccount(abc.ABC):

  def __init__(self, account_number, account_holder, initial_deposit):
    self.account_number = account_number
    self.account_holder = account_holder
    self.balance = 0
    self.deposit(initial_deposit)

  @abc.abstractmethod
  def deposit(self, amount):
    pass
  @abc.abstractmethod
  def withdraw(self, amount):
    pass


class SavingsAccount(BankAccount):

  def deposit(self, amount):
    if amount > 0:
      self.balance += amount

  def withdraw(self, amount):
      deduction = 0.05 * amount
      self.balance -= (amount + deduction)

if __name__ == "__main__":
  my_account = BankAccount("92338723", "John Smith", 5000)
  my_account.withdraw(500)
  print(my_account.balance)
```

- It won't print since the withdraw method is an abstract method
- 0

The code will throw an Error since it can't instantiate an object with abstract methods

4475

Done